Three Cases Where Using a ViewModel Is Necessary

A **ViewModel** is a class designed to carry the specific data required by a View. It helps separate the presentation layer from the database entities and application logic.

There are three important cases where using a ViewModel is necessary or strongly recommended:

1. Combining Data from Multiple Models

Sometimes, a single View needs data from multiple entities. Instead of passing multiple models separately to the View, we can create a ViewModel that combines the required data into one object.

Example: A Department Information page that displays the department name, a dropdown list of students older than 25, and the department state (Main or Branch).

Benefit: It organizes the data and makes it easier for the View to display everything it needs.

2. Security and Data Protection

A ViewModel helps prevent exposing sensitive or unnecessary properties of database entities to the client.

Example: A user profile page should display the user's name and email without exposing sensitive properties such as the password hash, internal permissions, or administrative fields.

By defining only the properties required by the ViewModel, we reduce the risk of accidentally exposing sensitive data or allowing users to submit values they should not control.

Benefit: It improves data protection and helps prevent overposting vulnerabilities when combined with proper server-side validation and authorization.

3. Customizing Data for Presentation

A View may need calculated, filtered, or formatted data that does not exist directly as a property in a database entity.

Example: A Department ViewModel can contain the department name, a list of students whose ages exceed 25, and a calculated department state based on the total number of students.

Benefit: It keeps presentation-specific calculations and data preparation separate from the database entities, making the application easier to maintain and test.

Conclusion

ViewModels are useful when a View needs combined data, when sensitive entity properties must be protected, or when the displayed data requires filtering and calculation. They improve separation of concerns, maintainability, and control over the data exchanged between the application and its Views.

SELF-STUDY REPORT

1. Lazy Loading vs. Eager Loading

Entity Framework Core provides strategies for loading related data. The right choice depends on which related entities are required and the number of database queries.

Eager Loading

Eager loading retrieves related data as part of the initial query. Include() and ThenInclude() are commonly used.

```
var department = await _context.Departments
    .Include(d => d.Students)
    .FirstOrDefaultAsync(d => d.Id == id);
```

Use case: A Department Details page must show the department and its students immediately. Eager loading makes the required relationship explicit and avoids a separate query for each department when used appropriately.

Lazy Loading

Lazy loading fetches related data when a navigation property is accessed. It requires suitable configuration, commonly lazy-loading proxies and virtual navigation properties.

```
services.AddDbContext<ApplicationDbContext>(options =>
    options.UseLazyLoadingProxies()
           .UseSqlServer(connectionString));
```

Use case: Related data is only occasionally needed, so the application defers loading until that data is accessed. Be aware that property access may trigger database queries.

Comparison and N+1

Aspect	Eager Loading	Lazy Loading
When it loads	With the initial query	When navigation property is accessed
Visibility	Explicit in query	May be hidden in property access
Risk	May load unnecessary data	Can trigger many extra queries
Useful when	Related data is known to be needed	Related data is needed conditionally

The N+1 problem occurs when one query loads a collection and then additional queries are executed for each item. For example, loading 50 departments and lazily accessing each department's students can cause one query plus up to 50 more. Eager loading or projection with Select() may be better.

2. “No Prior Definition” — Explicitly Define Ambiguous Relationships

“No Prior Definition” is not a standard standalone ASP.NET Core term. In the EF Core context, a useful interpretation is to avoid assumptions when a relationship or behavior needs to be clear. Conventions infer many relationships, but ambiguous models should be configured explicitly.

For example, explicitly connect Student.DepartmentId to Department.Students:

```
modelBuilder.Entity<Student>()
    .HasOne(s => s.Department)
    .WithMany(d => d.Students)
    .HasForeignKey(s => s.DepartmentId);
```

This documents the intended relationship and helps avoid EF Core inferring an unintended relationship or expecting a column such as DepartmentId1. If the instructor meant a different lecture-specific concept by this phrase, confirm its exact meaning.

3. Security in APIs

API security protects endpoints, data, and operations from unauthorized access. Hiding a button in the UI is not security; protection must be enforced on the server.

Authentication vs. Authorization

- Authentication answers: Who is the caller? Examples include cookies in browser applications and bearer tokens in APIs.
- Authorization answers: Is the caller allowed to perform this operation? It may be role-based, policy-based, or resource-based.

Best Practices

- Require authentication for protected endpoints and apply authorization policies or roles.
- Validate incoming data on the server; never trust values supplied by the client.
- Use HTTPS and protect signing keys, connection strings, and secrets.
- Do not return password hashes, internal fields, or unnecessary personal data.
- Use DTOs or ViewModels to limit fields clients can submit or receive and reduce overposting risks.
- Use sensible error handling, logging, rate limiting where appropriate, and least-privilege database permissions.

Example

```
[Authorize]
[HttpGet]
public IActionResult GetProfile()
{
    return Ok();
}
```

[Authorize] relies on correctly configured authentication and authorization services and middleware; the attribute alone does not configure the security system.

4. AutoMapper

AutoMapper maps values between objects, commonly between database entities and DTOs or ViewModels. It reduces repetitive property assignment but does not replace validation, authorization, or careful API design.

- Map an entity to a response DTO without exposing every entity property.
- Map an input DTO to an entity when creating or updating data.
- Centralize mapping rules and make them easier to test.

Example

```
public class StudentDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
}

public class StudentProfile : Profile
{
    public StudentProfile()
    {
        CreateMap<Student, StudentDto>();
    }
}
```

A Profile defines mapping rules. Register AutoMapper according to the package version, inject IMapper, and call Map when needed. For updates, map only permitted fields, validate input, and enforce authorization; do not blindly map client-controlled data to sensitive entity properties.

5. CQRS: Read ViewModel and Write ViewModel (Bonus)

CQRS means Command Query Responsibility Segregation. It separates operations that change state (commands) from operations that read data (queries). The principle can be used simply by defining different models for reading and writing; a full CQRS architecture may be more complex.

Read ViewModel

A Read ViewModel is shaped for display and may combine fields, calculated values, or related data.

Write ViewModel

A Write ViewModel contains only the input fields required for a specific operation, such as creating a student.

Example

```
public class StudentReadViewModel
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string DepartmentName { get; set; } = string.Empty;
}

public class CreateStudentViewModel
{
    public string Name { get; set; } = string.Empty;
    public int Age { get; set; }
```

```
    public int DepartmentId { get; set; }
}
```

The read model contains DepartmentName for display, while the write model accepts only fields needed to create a student. This improves clarity and reduces overposting risks. Separate ViewModels alone do not make a complete CQRS architecture.

6. Data Seeding in EF Core

Data seeding provides initial data for an application or database. It is useful for stable lookup values, reference data, and predictable development or testing scenarios.

Model-managed data with HasData

```
modelBuilder.Entity<Department>().HasData(
    new Department
    {
        Id = 1,
        Name = "IT",
        MgrName = "Ahmed"
    });
```

HasData is managed through EF Core migrations. Keys and values should be specified explicitly. After changing seed entries, create and apply a migration as appropriate.

Custom initialization

For dynamic setup, such as development test data or checking whether records already exist, initialization logic can use a DbContext. Avoid duplicate inserts and coordinate initialization carefully for production. Do not place secrets in seed data; use supported identity APIs for user accounts and passwords.